Résultats des scénarios — Communication architecture for order processing

OrderFlow — Experimental Evidence · Répétitions 1 (2026-11-08) et 2 (2026-11-09) · Statut : verdicts provisoires ou définitifs selon l'hypothèse

Executive conclusion

Les mesures ne confirment pas la décision candidate ADOPT EVENT-DRIVEN (ADR-001). L'asynchrone l'emporte nettement sur la résilience (H3a, H3b, H3c : supportées, définitives) et l'idempotence tient à faible coût (H4 : supportée, définitive). En revanche, H1 (pic) est non concluante avec une tendance « rejetée » : sur ce banc d'essai, le synchrone tient plus longtemps et se dégrade moins à 500 cmd/s, et aucun run async exploitable n'a pu être obtenu. H5 est rejetée : l'asynchrone produit 100 % de traces complètes. H2 est rejetée à titre provisoire (écart de 9,9 % contre un seuil de 20 %).

Evidence base : deux répétitions par scénario (S1, S2, S3, S4, S4b, S5), un seul hôte, une base PostgreSQL partagée. La répétition 2 de S1 et S2 a été exécutée sur une machine ralentie : elle est non concluante pour H1 et H2. Une répétition 3 en environnement propre est nécessaire (PC10).

Verdicts consolidés (deux répétitions)

Hypothèse	Verdict	Justification
H3a (Shipping arrêté)	**Supportée — définitive**	async : 100 % des commandes de la panne terminées aux deux répétitions ; sync : ≈ 2 830 confirmées jamais complétées
H3b (Payment arrêté)	**Supportée — définitive**	async : 0 perdue et 100 % en état final aux deux répétitions ; sync : ≈ 2 975 bloquées
H3c (Payment lent)	**Supportée — définitive**	async : 0 abandon, réponse p95 de 3 et 50 ms ; sync : 87,6 % et 87,2 % d'abandons

Hypothèse	Verdict	Justification
H4 (doublons)	**Supportée — définitive** (réserve : surcoût lu sur la médiane)	I7 = 0 avec protection ; témoins à 1 199 et 1 200 ; médianes identiques avec et sans doublons dans les deux variantes
H5 (traces)	**Rejetée — définitive**	async : 100 % de traces complètes aux deux répétitions (la mesure sync de la rép. 2 a échoué, sans effet sur ce verdict, qui ne dépend que de B)
Exigence S1 — sync	**Respectée — définitive**	68 et 71 ms
Exigence S1 — async	**Respectée — provisoire (rép. 1)** ; rép. 2 non concluante (environnement dégradé)	77 ms à la rép. 1 ; 1 660 ms à la rép. 2. Répétition 3 en environnement propre
H2 (latence)	**Rejetée — provisoire (rép. 1)** ; rép. 2 non concluante (environnement dégradé)	Rép. 1 : écart de 9,9 % (146 contre 162 ms), sous le seuil de 20 %. Rép. 2 : écart de 95 % dû au ralentissement de la machine pendant le run async. Répétition 3 en environnement propre
H1 (pic)	**Non concluante, tendance rejetée — provisoire (rép. 1)** ; rép. 2 non concluante	Rép. 1 : sync valide (70,6 %, 3,7 s), async invalide (43,8 %, 371 s, indicatif) ; rép. 2 : sync 54,6 % en environnement dégradé, aucun run async exploitable. Répétition 3 en environnement propre, et un moyen d'obtenir un run async exploitable

1. S1 — Charge normale, 100 cmd/s (exigence S1, H2, H5)

Répétition 1, 2026-11-08. Profil `s1` : préchauffage 1 min à 50 cmd/s (non compté), puis 3 min à 100 cmd/s (phase `measure`). Traces échantillonnées à 10 % (PC3).

Run	Fichiers
sync	`results/sync/s1/rep-1/`
async	`results/async/s1/rep-1/`

Les deux runs sont **valides** (aucune itération perdue par k6).

Mesures (phase `measure`)

Mesure	sync	async
Commandes mesurées	17 913	17 976
Débit reçu / accepté	99,5 / 96,6 cmd/s	99,9 / 96,7 cmd/s
Acceptation p50 / p95 / p99	53 / **68** / 318 ms	53 / **77** / 167 ms
Complétion p50 / p95 / p99	98 / **146** / 615 ms	100 / **162** / 334 ms
Réponse HTTP p95 (k6)	154 ms	11 ms
Abandons côté client	0	0
Drainage après le run	0 s (rien en cours)	0 s (rien en cours)
Traces complètes (commandes échantillonnées)	1 846 / 1 846 (100 %)	1 780 / 1 780 (100 %)
Invariants	OK	2 violations de I9 (`confirmed_at < accepted_at`), artefact de mesure (voir lecture, point 5)

Le débit accepté est inférieur au débit reçu des 3 % de paiements refusés (D12), jamais « acceptés » : c'est attendu.

Sources : `run.json` , `latency.json` (`measure`), `invariants.json` , `traces.json` de chaque dossier.

Lecture

1. **Exigence S1 respectée par les deux variantes** : acceptation p95 de 68 et 77 ms, très loin de 1 s ; aucun travail accumulé à la fin du run.
2. **Latence** : le synchrone complète un peu plus vite au p95 (146 contre 162 ms, soit 9,9 % de moins), mais l'asynchrone a une queue plus courte au p99 (334 contre 615 ms en complétion, 167 contre 318 ms en acceptation). Les latences simulées (50 + 10 + 20 + 10 ms) dominent dans les deux cas : à 100 cmd/s, aucune variante ne force.
3. **Réponse au client** : 11 ms en async (`202` dès l'enregistrement) contre 154 ms en sync (réponse après toute la chaîne).
4. **Traces** : 100 % de traces complètes dans les deux variantes ; la propagation du contexte à travers l'outbox (AS9) fonctionne. Le coût est dans l'effort d'instrumentation (code manuel, défauts trouvés en lisant les traces : M3, C6), pas dans le résultat.
5. **Violations de I9 en async** : 2 commandes sur 20 975 (run entier) ont un `confirmed_at` antérieur à leur `accepted_at`. Ce n'est pas une incohérence métier (I1 à I8 respectés, l'ordre réel est correct) : `now()` donne l'heure du début de la transaction, et deux événements de la même commande traités en même temps par Order peuvent ainsi écrire des heures inversées. Défaut de mesure connu, non corrigé : `docs/hypotheses-assumption.md`, §4.

Verdicts

Hypothèse	Verdict	Justification
Exigence S1 (p95 d'acceptation < 1 s, pas d'accumulation)	**Respectée par A et B — provisoire** (1 répétition sur 2)	68 et 77 ms ; drainage nul
H2 (sync : complétion p95 au moins 20 % plus faible, et acceptation p95 < 1 s)	**Rejetée — provisoire** (1 répétition sur 2)	Écart de 9,9 % (146 contre 162 ms), sous le seuil de 20 % ; la condition d'acceptation de A est remplie
H5 (async < 99 % de traces complètes, sync ≥ 99 %)	**Rejetée — provisoire** (1 répétition sur 2)	async : 100 % (1 780 / 1 780)

2. S4b — Shipping arrêté 30 s (H3a)

Répétition 1, 2026-11-08. Charge : 100 cmd/s ; Shipping arrêté 30 s au milieu du run (profil `s4b`, `common/benchmarks/profiles.js`). Traces échantillonnées à 10 % (PC3).

Run	Panne (UTC)	Fichiers
async	07:54:18 → 07:54:48	`results/async/s4b/rep-1/`
sync	08:00:57 → 08:01:27	`results/sync/s4b/rep-1/`

Les deux runs sont **valides** (aucune itération perdue par k6, panne de 30 s à l'heure prévue).

Mesures (commandes reçues pendant la panne, phase `fault`)

Mesure	sync	async
Commandes	3 000	3 000
Perdues (envoyées par k6, absentes de la base)	0	0
Acceptation p95	56 ms	58 ms
Réponse HTTP p95 (k6)	99 ms	3 ms
Abandons côté client (délai de 5 s)	2	0
Confirmées sans être complétées (ni expédition ni notification)	**2 841**	**0**
Complétion p95 (commandes complétées)	104 ms (≈ 100 commandes seulement, en bord de fenêtre)	30,2 s (toutes)
Fin du drainage	« plus de progression depuis 30 s » : commandes bloquées	toutes les commandes terminées
Invariants (tout le run)	I11 violé : 2 967 commandes confirmées jamais complétées	OK

Sources : `run.json` (validité, panne, k6), `latency.json` (`byPhase.fault`), `invariants.json` de chaque dossier.

Lecture

1. **Accepter les commandes : aucune différence.** Avec D5 (confirmation dès que le stock est réservé, avant Shipping), les deux variantes acceptent et confirment normalement pendant la panne (acceptation p95 ≈ 57 ms des deux côtés). La formulation de H3 dans l'énoncé (« continuer à **accepter** des commandes ») ne départage pas les styles dans ce scénario.
2. **Terminer les commandes : différence nette.** En async, les demandes d'expédition attendent dans la file de Shipping et sont traitées à son retour : 3 000 commandes sur 3 000 vont au bout (complétion p95 ≈ 30 s, la durée de la panne). En sync, l'appel à Shipping échoue aussitôt et, sans nouvelle tentative, la commande reste confirmée sans expédition ni notification : 2 841 commandes sur 3 000 ne seront jamais complétées (client débité, rien expédié).
3. **Origine de l'écart.** L'async met en file **sans mécanisme ajouté** ; le sync aurait besoin d'**ajouter** un mécanisme (nouvelle tentative, file d'attente) pour obtenir le même résultat. C'est un avantage réel du style, mais il dépend de la règle d'équité « budget de nouvelles tentatives 0 » (AS5) : avec un budget non nul dans les deux variantes (expérience séparée), l'écart serait réduit.

Verdict

Hypothèse	Verdict	Justification
H3a (async : ⩾ 99 % des commandes de la panne terminées en moins de 2 min après le retour de Shipping)	**Supportée — provisoire** (1 répétition sur 2)	async : 100 % terminées, drainage terminé avant la fin du run (90 s après le retour de Shipping) ; sync : ≈ 3 % complétées. Reste à mesurer précisément le délai après le retour (`max(completed_at)` − fin de panne) et à faire les répétitions 2 et 3.

3. S4 — Payment arrêté 30 s (H3b)

Répétition 1, 2026-11-08. Charge : 100 cmd/s ; Payment arrêté 30 s au milieu du run (profil `s4`). Traces échantillonnées à 10 % (PC3).

Run	Panne (UTC)	Fichiers
sync	08:16:30 → 08:17:00	`results/sync/s4/rep-1/`
async	08:25:01 → 08:25:31	`results/async/s4/rep-1/`

Les deux runs sont **valides** (aucune itération perdue par k6, panne de 30 s à l'heure prévue).

Mesures

Mesure	sync	async
Commandes reçues pendant la panne (phase `fault`)	3 000	3 000
Perdues (envoyées par k6, absentes de la base)	**0**	**0**
Terminées (`CONFIRMED` ou `REJECTED`)	**24** (premières millisecondes de la fenêtre, avant l'arrêt effectif de Payment)	**3 000** (2 847 confirmées, 88 paiements refusés, 65 stocks insuffisants)
Bloquées (sans état final après drainage)	**2 976** en `RECEIVED`	**0**
Réponse au client pendant la panne (k6, p95)	`502` en 14 ms (3 abandons)	`202` en 4 ms (0 abandon)
Acceptation p95 des commandes de la panne	— (jamais acceptées)	30,4 s (attente du retour de Payment)
Acceptation p95 des commandes reçues après la panne (phase `after`)	**58 ms**	**5,6 s**

Mesure	sync	async
Fin du drainage	« plus de progression depuis 30 s » : commandes bloquées	toutes les commandes terminées avant la fin du run
Invariants (tout le run)	I10 violé : 3 112 commandes jamais terminées	I9 : 1 violation (artefact de `now()`, `docs/hypotheses-assumption.md` §4)

Sources : `run.json` (validité, panne, k6, commandes perdues), `latency.json` (`byPhase.fault`, `byPhase.after`), `invariants.json` de chaque dossier.

Lecture

1. **Pendant la panne**, async **prend en compte** toutes les commandes (`202`) et les termine toutes au retour de Payment ; sync les **refuse** (`502`) et, sans nouvelle tentative (AS5), les laisse bloquées en `RECEIVED` pour toujours. Aucune variante ne perd de commande (chacune est enregistrée avant l'appel à Payment ou l'envoi de l'événement), et aucune ne confirme sans paiement (I1 à I8 respectés des deux côtés).
2. **Coût caché de l'async, après la panne** : les commandes en attente sont traitées dans l'ordre, avant les nouvelles. Les commandes arrivées juste après la panne attendent 5,6 s au p95 (contre 58 ms en sync) : pendant un moment, l'async dépasse le critère p95 < 1 s pour des commandes qui n'ont pas subi la panne. Le redémarrage de Payment à froid (connexions à la base, C2) contribue aussi.
3. **Compromis** : l'async sauve les commandes de la panne, au prix d'un ralentissement temporaire des suivantes ; le sync reste rapide pour la suite, au prix de commandes abandonnées pour le client (erreur `502`, commande bloquée). Ce compromis n'entre pas dans le verdict de H3b ; il doit figurer dans l'ADR (*Trade-offs*).
4. Comme en S4b, l'écart dépend de la règle « budget de nouvelles tentatives 0 » : le sync aurait besoin d'un mécanisme ajouté pour rattraper les commandes de la panne.

Verdict

Hypothèse	Verdict	Justification
H3b (async : 0 perdue et ⩾ 99 % des commandes de la panne dans un état final moins de 2 min après le retour de Payment)	**Supportée — provisoire** (1 répétition sur 2)	async : 0 perdue, 3 000 / 3 000 en état final, drainage terminé avant la fin du run (90 s après le retour de Payment) ; sync : 2 976 bloquées sur 3 000. Reste à mesurer précisément le délai après le retour et à faire les répétitions 2 et 3.

4. S3 — Payment lent : 2 s pendant 60 s (H3c)

Répétition 1, 2026-11-08. Charge : 100 cmd/s ; latence de Payment portée à 2 s pendant 60 s au milieu du run par `POST /admin/chaos` (profil `s3`), sonde `GET /orders/:id` une fois par seconde. Délai client de 5 s (PC2), VUs préparés d'avance (PC6). Traces échantillonnées à 10 % (PC3).

Run	Lenteur (UTC)	Écart à l'heure prévue (pose / levée)	Fichiers
sync	voir `run.json` (`faultTiming`)	10 ms / 8 ms	`results/sync/s3/rep-1/`
async	09:08:43 → 09:09:43	voir `run.json` (`faultTiming`)	`results/async/s3/rep-1/`

Les deux runs sont **valides** (aucune itération perdue par k6, lenteur de 60 s à l'heure prévue).

Mesures (phase `fault`, sauf mention)

Mesure	sync	async
Requêtes envoyées par k6	5 989	6 000

Mesure	sync	async
Abandons côté client (sans réponse en 5 s)	**5 247 (87,6 %)**	**0**
Réponse HTTP p95 (k6)	**5 002 ms** (le délai client)	**3 ms**
Sonde `GET /orders/:id` p95	3 ms	1,6 ms
Commandes allées au bout (état final)	**625** (≈ 10 acceptées par seconde)	**6 000**
Commandes bloquées pour toujours	**5 364** (`RECEIVED`)	**0**
Perdues (absentes de la base)	0	0
Acceptation p95	4,7 s (sur les 625 acceptées)	**39,8 s** (toutes, en attente dans la file)
Acceptation p95 des commandes reçues **après** la lenteur (phase `after`)	**176 ms**	**7,5 s**
Invariants (tout le run)	I10 : 5 364 commandes jamais terminées	I9 : 2 (artefact de `now()`, `docs/hypotheses-assumption.md` §4)

Sources : `run.json` (validité, `faultTiming`, k6 par phase), `latency.json` (`byPhase.fault`, `byPhase.after`), `invariants.json` de chaque dossier.

Lecture

1. **La différence n'est pas le débit, c'est qui subit l'attente.** Avec Payment à 2 s et au plus 64 traitements en parallèle, la capacité de Payment est d'environ 32 commandes par seconde dans les deux variantes, pour 100 reçues. En **sync**, c'est le **client** qui attend : 87,6 % des requêtes dépassent 5 s et presque toutes les commandes de la fenêtre restent bloquées (Order abandonne aussi ses appels à Payment au bout de 5 s, attente d'une connexion libre comprise). En **async**, c'est la **file** qui attend : le client a sa réponse en 3 ms, et toutes les commandes aboutissent, environ 40 s plus tard.

2. **Le sync gaspille la capacité de Payment** : ≈ 10 commandes acceptées par seconde au lieu de ≈ 32 possibles, car beaucoup d'appels attendent une connexion libre puis expirent sans avoir servi. L'async utilise toute la capacité et rattrape tout.
3. **Même compromis qu'en S4** : après la lenteur, le sync repart aussitôt (176 ms), l'async fait attendre les commandes suivantes derrière sa file (7,5 s au p95, au-delà du critère de 1 s).
4. **Les lectures ne sont gênées dans aucune variante** (sonde à quelques millisecondes) : Order n'est pas saturé en CPU, l'attente est ailleurs (connexions vers Payment en sync, file en async).

Verdict

Hypothèse	Verdict	Justification
H3c (async : réponse HTTP p95 < 200 ms et 0 abandon ; sync : plus de 50 % d'abandons)	**Supportée — provisoire** (1 répétition sur 2)	async : 3 ms et 0 abandon ; sync : 87,6 % d'abandons. Compromis hors verdict : acceptation à ≈ 40 s pendant la lenteur et 7,5 s juste après en async.

5. S2 — Pic de trafic (H1)

Répétition 1, 2026-11-08. Profil `s2` : préchauffage 1 min à 50 cmd/s, paliers de 1 min à 100, 200, 300, 400 cmd/s, puis 5 min à 500 cmd/s (fenêtre du verdict). Délai client de 5 s (PC2), 2 500 VUs préparés d'avance (PC6). Traces échantillonnées à 10 % (PC3).

Run	Validité	Fichiers
async	**INVALIDE** : 9 310 itérations perdues par k6 (4,38 % ⩾ 1 %)	`results/async/s2/rep-1/`
sync	**VALIDE** : 294 itérations perdues (0,14 %)	`results/sync/s2/rep-1/`
async, répétition 2	**INVALIDE** : 1,09 % d'itérations perdues (k6 préparé avec 3 000 VUs, marge de 20 %) ; VUs au plafond (2 999 / 3 000) mais ressources de k6 non relevées, donc non reclassable « valide sous saturation » (PC7)	`results/async/s2/rep-2/`

Run	Validité	Fichiers
async, répétition 3	Classée « valide sous saturation » par le script ; **invalide** après examen (k6 à 1 307 % de CPU, préchauffage dégradé)	`results/async/s2/rep-3/`

Statut. Le run async, invalide, est à refaire (CLAUDE.md §8) ; ses chiffres sont **indicatifs** : ils ne servent pas au verdict de H1, mais orientent la suite. Le run sync est valide.

async, répétition 1 (invalide) : mesures par palier

Palier (offert)	Reçu/s	Accepté/s	Acceptation p95	Réponse HTTP p95 (k6)	Abandons côté client
50 (préchauffage)	50,0	48,6	201 ms	55 ms	0
100	99,9	97,1	218 ms	99 ms	0
200	199,2	192,8	**10,9 s**	**1,8 s**	0
300	299,1	290,1	**403 s**	1,4 s	0
400	**172,3**	167,0	405 s	5,0 s	15 275
500 (fenêtre du verdict)	**225,7**	**219,0**	**371 s**	5,1 s	137 380

« Accepté/s » compte les commandes reçues pendant le palier qui ont fini par être acceptées, éventuellement des minutes plus tard. Drainage après la charge : 273 s, toutes les commandes terminées (aucune bloquée). Invariants : I1 à I8 respectés ; 3 violations de I9 (artefact de `now()`, `docs/hypotheses-assumption.md` §4).

CPU maximal des conteneurs pendant le run (`resources.json`, échantillons de `docker stats` ; au-delà de la limite = au plafond, artefact d'échantillonnage) :

Conteneur	CPU max	Limite
PostgreSQL	**222 %**	200 %
Order	**116 %**	100 %
Payment	98 %	100 %
Inventory	91 %	100 %
RabbitMQ	164 %	200 %

Conteneur	CPU max	Limite
Shipping / Notification	78 % / 72 %	100 %

Critères de H1 sur le palier à 500 cmd/s, **à titre indicatif** : débit accepté 219,0 / 500 = 43,8 % (seuil ⩾ 95 %) ; acceptation p95 371 s (seuil < 1 s) ; violations de I1 : 0 (seuil 0).

async, répétition 2 (invalide de justesse) : mesures par palier

Palier (offert)	Reçu/s	Accepté/s	Acceptation p95	Réponse HTTP p95 (k6)	Abandons côté client
50 (préchauffage)	50,0	48,7	65 ms	7 ms	0
100	99,6	96,8	105 ms	26 ms	0
200	199,1	193,5	**1,7 s**	452 ms	0
300	299,1	290,3	**375 s**	983 ms	0
400	399,0	386,3	375 s	1,9 s	0
500 (fenêtre du verdict)	**270,9**	**262,8**	**370 s**	5,0 s	130 268

Drainage : 334 s, toutes les commandes terminées. Invariants : I1 à I8 respectés, 2 violations de I9 (artefact de `now()`). CPU maximal : PostgreSQL 213 % (limite 200 %, saturé), Order 124 % (saturé), Inventory 99 %, Payment 96 %, RabbitMQ 157 %, Shipping 82 %, Notification 81 %.

Critères de H1 à 500 cmd/s : débit accepté 262,8 / 500 = **52,6 %** (seuil ⩾ 95 %) ; acceptation p95 **370 s** (seuil < 1 s) ; I1 : 0. Les deux répétitions async disent la même chose : décrochage dès 200 cmd/s, acceptation à six minutes à partir de 300 cmd/s, base partagée et Order saturés. Le ratage des seuils est massif ; selon PC7, il resterait valable avec 1 % de charge en moins, à condition de prouver que k6 n'était pas saturé : c'est l'objet de la répétition 3, avec relevé des ressources de k6.

async, répétition 3 : classée « valide sous saturation » par le script, mais **à considérer comme invalide**

`results/async/s2/rep-3/` : 52 919 itérations perdues (24,98 %), VUs au plafond (3 000 / 3 000), 97,5 % des requêtes mesurées au délai client. Le script l'a classée « valide sous saturation » (PC7), mais l'examen montre que **k6 a lui-même saturé la machine** :

Indice	Valeur	Lecture
CPU de k6 (`poc-k6-async-s2-r3`)	**1 307 %** (≈ 13 cœurs ; Docker dispose de 10 CPU)	k6 a consommé toute la machine
Préchauffage (50 cmd/s)	397 abandons, HTTP p95 5 s, acceptation p95 10,9 s	Anormal dès le départ (7 à 55 ms aux répétitions 1 et 2) : machine surchargée avant la montée en charge
Conteneurs au-delà de leur limite	PostgreSQL 279 % (limite 200), Order 143 % (limite 100)	Sous cette contention, les relevés de `docker stats` ne sont plus fiables
Reçu au palier à 500	66,8 cmd/s	Sans commune mesure avec les répétitions 1 (226) et 2 (271)

Ce run mesure une machine surchargée par k6, pas le comportement de l'async. Il n'est **pas utilisé**. Le critère de santé de k6 de PC7 (mémoire seule) l'a laissé passer : correction proposée, non appliquée, dans `docs/decisions-conception.md` (PC7). Cause probable : avec 3 000 VUs qui atteignent presque tous le délai de 5 s, k6 ouvre et ferme des centaines de connexions par seconde ; la marge de VUs de 20 % a aggravé sa consommation. Un autre programme a peut-être aussi chargé la machine dès le début (préchauffage dégradé).

sync, répétition 1 (valide) : mesures par palier

Palier (offert)	Reçu/s	Accepté/s	Acceptation p95	Réponse HTTP p95 (k6)	Abandons côté client
50 (préchauffage)	50,0	48,5	63 ms	141 ms	0
100	99,6	96,5	57 ms	127 ms	0
200	199,1	192,8	75 ms	178 ms	0
300	298,0	288,7	**619 ms**	**1,4 s**	0
400	**344,9**	335,1	**5,2 s**	5,0 s	8 820
500 (fenêtre du verdict)	**363,8**	**352,8**	**3,7 s**	5,0 s	98 177

Drainage après la charge : 1 s, toutes les commandes terminées (aucune bloquée). Invariants : tous respectés.

CPU maximal des conteneurs (`resources.json`) : Order 113 % (limite 100 %, **saturé**), PostgreSQL 146 % (limite 200 %, chargé mais pas saturé), Payment 67 %, Inventory 62 %, Notification 48 %, Shipping 46 %.

Comparaison sync / async (répétition 1)

Palier	Acceptation p95 sync	Acceptation p95 async (invalide)	HTTP p95 sync	HTTP p95 async
100 cmd/s	57 ms	218 ms	127 ms	99 ms
200 cmd/s	**75 ms**	**10,9 s**	178 ms	1,8 s
300 cmd/s	**619 ms**	**403 s**	1,4 s	1,4 s
400 cmd/s	5,2 s	405 s	5,0 s	5,0 s
500 cmd/s	**3,7 s**	**371 s**	5,0 s	5,1 s

Palier à 500 cmd/s	sync	async (invalide)	Seuil de H1
Débit accepté / offert	**352,8 / 500 = 70,6 %**	219,0 / 500 = 43,8 %	⩾ 95 %
Acceptation p95	**3,7 s**	371 s	< 1 s
Violations de I1	0	0	0
Drainage après la charge	**1 s**	273 s	—
Commandes bloquées	0	0	—

Conteneur (limite)	CPU max sync	CPU max async (invalide)
PostgreSQL (200 %)	146 %	**222 % (saturé)**
Order (100 %)	**113 % (saturé)**	**116 % (saturé)**
Payment (100 %)	67 %	98 %

Conteneur (limite)	CPU max sync	CPU max async (invalide)
Inventory (100 %)	62 %	91 %
Shipping / Notification (100 %)	46 % / 48 %	78 % / 72 %
RabbitMQ (200 %)	—	164 %

Lecture (provisoire)

1. **Vue d'ensemble.** Aucune des deux variantes ne tient 500 cmd/s sur ce banc d'essai. Le **sync tient plus longtemps et se dégrade moins** : il reste sous le critère p95 < 1 s jusqu'à 300 cmd/s (l'async décroche dès 200), accepte 61 % de commandes de plus à 500 cmd/s (353 contre 219 par seconde), et sa latence reste bornée à quelques secondes quand celle de l'async s'emballe à plusieurs minutes. Les deux se dégradent différemment : le sync échoue côté client (abandons au-delà de 5 s) mais le serveur continue et tout est terminé 1 s après la charge ; l'async répond puis accumule, et met 4,5 min à se vider. À la question de l'énoncé (« quelle architecture se dégrade le plus progressivement ? »), ce run répond **le synchrone**.
2. **Goulots différents.** En sync, le goulot est **Order seul** (il orchestre toute la chaîne sur un cœur, comme dans l'essai C5), la base et les autres services gardent de la marge. En async, **la base partagée sature en plus d'Order**, et tous les services travaillent davantage (Payment 98 % contre 67 %, Inventory 91 % contre 62 %), RabbitMQ consommant en plus 1,6 cœur : pour le même travail métier, l'async consomme nettement plus de ressources (outbox, idempotence, publication et consommation, une transaction par événement). Le goulot du sync se lèverait en ajoutant des instances d'Order ; celui de l'async demanderait de répartir la base.
3. **L'async décroche dès 200 cmd/s** sur ce banc d'essai : la réponse HTTP passe à 1,8 s et l'acceptation à 10,9 s au p95 ; à 300 cmd/s, les commandes sont acceptées plus de 6 minutes après leur arrivée. Le run sync (valide) décroche, lui, vers 300 cmd/s (acceptation p95 619 ms, réponse HTTP 1,4 s), puis atteint 5,2 s d'acceptation à 400 cmd/s.
4. **Le goulot principal est PostgreSQL**, partagé par tous les services et saturé ; **Order** sature en même temps. Pour une commande, l'async fait beaucoup plus d'écritures que le sync : une ligne d'outbox par événement puis une transaction pour la marquer publiée (AS7), une ligne `processed_events` par événement reçu, une transaction par événement traité, avec environ six événements par commande. Order traite aussi la requête HTTP et trois à quatre événements par commande sur un seul cœur.

5. **Ce coût est en partie celui du style** : l'outbox et l'idempotence sont nécessaires pour tenir l'invariant en asynchrone (question de S5). Il apparaît ici en CPU de base de données.
6. **Il dépend aussi de nos choix d'implémentation** (par exemple la transaction supplémentaire qui marque l'outbox publiée) : un allègement serait une expérience séparée et documentée, pas une correction silencieuse.
7. **Limites du banc d'essai** : une base PostgreSQL unique partagée par les cinq services (menace à la validité, `docs/hypotheses-assumption.md` §7), une seule machine, un cœur par service. Dans une architecture réelle avec une base par service, le goulot serait réparti. Conclure sur **ce banc d'essai**, pas sur l'événementiel en général.
8. **La sûreté tient même en pleine saturation** : aucune confirmation sans paiement ni stock (I1 à I8 respectés), aucune commande perdue ni bloquée.
9. **Contraire à la décision candidate** (ADOPT EVENT-DRIVEN, ADR-001) : à confirmer par la répétition valide et par le run sync avant toute conclusion.

Point de protocole à régler (validité)

Les itérations perdues viennent d'un système qui ne répond plus : à partir de 400 cmd/s, presque toutes les requêtes atteignent le délai client de 5 s, et k6 a alors besoin d'exactement débit × délai = 2 500 VUs, sans aucune marge (PC6). Toute variante qui sature déborde k6 de la même façon et rend le run invalide par la règle des 1 %. À décider avant de refaire S2 : marge de VUs au-delà de débit × délai (le plafond actuel de 3 000 VUs en laisse), ou traitement explicite de ce cas dans la règle de validité.

Verdict

Hypothèse	Verdict	Justification
H1 (async tient les trois seuils à 500 cmd/s, sync en rate au moins un)	**Non concluante pour l'instant — tendance : rejetée**	Runs async invalides (répétitions 1 et 2 : 4,38 % puis 1,09 % d'itérations perdues, système saturé ; répétition 3 : k6 a saturé la machine). Aucun run async utilisable pour le verdict : la machine ne peut pas porter à la fois k6 et un système saturé à 500 cmd/s (limite du banc d'essai, A-15). Comment obtenir un run async exploitable reste à décider. Répétition 2 : 52,6 % accepté, acceptation p95 370 s. Indication : l'async rate deux seuils sur trois (43,8 % accepté, acceptation p95 371 s) ; le sync (valide) en rate aussi deux (70,6 %, 3,7 s) mais va nettement plus loin. Résultat contraire à la décision candidate (ADR-001), obtenu sur un banc d'essai dont la base unique partagée sature en premier sous l'async.

6. S5 — Livraison multiple (H4)

2026-11-08. Profil `s5` : préchauffage 1 min à 10 cmd/s, puis 1 min à 10 cmd/s (phase `measure`). Traces à 100 % (PC3). Trois runs par variante : ***avec protection*** *(doublons, idempotence active),* ***référence*** *(`--no-duplicates` : sans doublons, pour le surcoût),* ***témoin*** *(`--witness` : doublons, protection désactivée, pour prouver que le test détecte*

les doublons). Doublons : en async, chaque événement est publié deux fois (`DUPLICATE_DELIVERY=2`) ; en sync, k6 renvoie chaque requête après la première réponse avec le même `Idempotency-Key` (PC5).

Run	Fichiers
async, répétition 1	`results/async/s5/rep-1/`, `rep-1-nodup/`, `rep-1-witness/`
async, répétition 2 (protection et référence)	`results/async/s5/rep-2/`, `rep-2-nodup/`
sync, répétition 1	`results/sync/s5/rep-1/`, `rep-1-nodup/`, `rep-1-witness/`

Tous les runs sont **valides**.

Mesures

Run	Commandes	Complétion p50 / p95	I7 (effets en double)	I1	Commandes en double (même requête client)	Doublons écartés
async, protection, rép. 1	600	— / 183 ms	**0**	0	0	8 317
async, référence, rép. 1	601	— / 1 393 ms (pics de la machine, voir lecture)	0	0	0	0
async, **témoin**, rép. 1	599	— / 344 ms	**1 199**	0	0	0
async, protection, rép. 2	—	**112** / 144 ms	—	—	—	8 333
async, référence, rép. 2	—	**113** / 378 ms	—	—	—	0
sync, protection, rép. 1	601	— / 171 ms	0	0	**0**	≈ 1 200

Run	Commandes	Complétion p50 / p95	I7 (effets en double)	I1	Commandes en double (même requête client)	Doublons écartés
sync, référence, rép. 1	600	— / 145 ms	0	0	0	(compteur faux, voir lecture, point 7)
sync, **témoin**, rép. 1	1 198	— / 154 ms	**0**	0	**1 199**	0

Sources : `run.json` (`validity` , `duplicates`), `latency.json` (`measure`), `invariants.json` de chaque dossier. Les cases « — » n'ont pas été relevées.

Lecture

1. **La protection fonctionne dans les deux variantes** : avec doublons, aucun effet en double en async (I7 = 0) et aucune commande en double en sync ; aucune violation de I1. **Les deux témoins prouvent que le test détecte les doublons** : I7 = 1 199 en async, 1 199 commandes en double en sync.
2. **Les doublons ne se manifestent pas de la même façon.** En async, ils dupliquent les **effets** sur la même commande (paiements, réservations, expéditions, notifications) : I7 les voit. En sync, un renvoi du client crée une **seconde commande** : I7 ne la voit pas, d'où la mesure « commandes en double par requête client ».
3. **Volume à traiter** : l'async écarte environ **7 doublons par commande** (chaque événement livré deux fois à chaque consommateur), le sync **1** (le renvoi du client). La protection async est plus sollicitée, et elle repose sur un mécanisme à maintenir dans chaque consommateur (`processed_events` dans la même transaction que l'effet).
4. **Surcoût de la protection en async : nul à la médiane** (complétion p50 112 ms avec doublons contre 113 ms sans, répétition 2). Écarter un doublon revient à une insertion refusée dans la même transaction.
5. **Le p95 à 10 cmd/s est dominé par des pics de la machine.** La phase mesurée ne compte que 600 commandes : le p95 ne regarde que les 30 plus lentes, et un arrêt passager de quelques secondes suffit à le faire grimper (référence async, répétition 1 : p95 de tranche à 2,6 s à 30 s et 2,3 s à 100 s, normal ailleurs). Dans les deux répétitions, la **référence** (sans doublons) a un p95 **plus élevé** que le run avec doublons ; elle passe toujours en second, sur des conteneurs recréés (le réglage des doublons change). Effet d'ordre probable, à vérifier en inversant l'ordre à la répétition 3.

6. **Sync, pour comparaison (hors verdict)** : surcoût de la protection +18 % au p95 (171 contre 145 ms, soit 26 ms, dans la zone de bruit). Le renvoi double le nombre de requêtes HTTP ; le second est servi depuis `idempotency_keys`.
7. **Défaut de mesure corrigé en cours de route** : le compteur « doublons écartés » de la référence sync (répétition 1) affichait la valeur du run précédent, les conteneurs n'étant pas redémarrés entre deux runs aux réglages identiques. `run-scenario.js` mesure désormais la différence du compteur entre le début et la fin du run (vérifié à la répétition 2 async : 8 333 puis 0).

Verdict

Hypothèse	Verdict	Justification
H4 (avec idempotence : I7 = 0, I1 = 0, surcoût de complétion p95 < 10 % ; le témoin doit montrer des doublons)	**Supportée — provisoire** (2 répétitions de la protection et de la référence, 1 du témoin)	I7 = 0 et I1 = 0 avec protection ; témoin I7 = 1 199 ; surcoût p95 négatif dans les deux répétitions (−86,8 %, −62 %), donc sous 10 %. **Réserve** : à 10 cmd/s, le surcoût se lit sur la médiane (0 %), le p95 étant dominé par les pics de la machine. Répétition 3 à faire en inversant l'ordre (référence d'abord).

7. Répétition 2 (2026-11-09) et verdicts consolidés

Ordre inversé par rapport à la répétition 1 (PC9) : async d'abord, puis sync. Tous les runs sont valides.

Mesures de la répétition 2, face à la répétition 1

Scénario	Mesure	sync rép. 1	sync rép. 2	async rép. 1	async rép. 2
S1	Acceptation p95	68 ms	71 ms	77 ms	**1 660 ms**
S1	Complétion p95	146 ms	150 ms	162 ms	**3 123 ms**

Scénario	Mesure	sync rép. 1	sync rép. 2	async rép. 1	async rép. 2
S1	Traces complètes	1 846 / 1 846	mesure échouée (`tracesExitCode` 2)	1 780 / 1 780	1 738 / 1 738
S2	Débit accepté à 500 cmd/s ; acceptation p95	70,6 % ; 3,7 s	54,6 % ; 4,0 s	invalide	pas de run exploitable
S3	Abandons côté client ; réponse HTTP p95	87,6 % ; 5 002 ms	87,2 % ; 5 002 ms	0 ; 3 ms	0 ; 50 ms
S4	Commandes de la panne terminées ; perdues	24 / 3 000 ; 0	27 / 3 000 ; 0	3 000 / 3 000 ; 0	2 998 / 2 998 ; 0
S4b	Confirmées sans être complétées ; commandes terminées (async)	2 841	2 825	3 000 / 3 000	3 000 / 3 000
S5	Avec protection : effets en double / commandes en double	0	0	I7 = 0	— (fait à la rép. 2 de S5)
S5	Témoin	1 199 commandes en double	1 199 commandes en double	I7 = 1 199	I7 = 1 200
S5	Complétion p50 avec protection / référence	—	112 / 113 ms	112 / 113 ms (rép. 2)	—

Fichiers : `results/<projet>/<scénario>/rep-2/` (et `rep-2-nodup/`, `rep-2-witness/` pour S5).

Environnement dégradé pendant les runs S1 et S2 de la répétition 2

Le run async S1 de la répétition 2 est nettement plus lent que celui de la répétition 1 (acceptation p95 1 660 ms contre 77 ms), avec des pics ponctuels (3,5 s vers 180 s) entre des périodes normales. Indices d'une **machine ralentie**, pas d'un changement du système : tous les conteneurs consomment plus de CPU pour le même travail (PostgreSQL 222 % contre 155 %, Notification 116 % contre 36 %), la base est saine (pas de lignes mortes, 17 Mo), k6 n'est pas en cause (1 cœur, 3 % de mémoire). Mêmes signes en S4b et S3 async (acceptation et phase « après » plus lentes), et en S2 sync (54,6 % accepté contre 70,6 %). Causes possibles : Mac sur batterie ou en économie d'énergie, chauffe du processeur, autres applications.

Décision de l'équipe du POC (2026-11-09). Ces runs restent **valides** selon nos critères et ne sont pas écartés après coup ; mais pour **H1 et H2**, la répétition 2 est déclarée **non concluante** (environnement dégradé), et le verdict garde les résultats de la **répétition 1**, provisoires. Une **répétition 3 dans un environnement propre** est nécessaire (PC10, `docs/decisions-conception.md`).

Les verdicts consolidés sont résumés en tête de document (Executive conclusion).